CSC 402 — MILESTONE 3-A

Jeel | Database Design

Due: Week 12 • 5 marks • One submission per team (PDF)

How to use this template

1. Fill every yellow cell. Replace the grey example rows with your own data and delete the grey hint lines.
2. Be brief and exact: short phrases, real names, real fields — no general explanations.
3. Design the database for your own app's features from Milestone 2. Copy a table as many times as you need.
4. Export as PDF and name it Team##_M3A_Database.pdf. Commit the same files to /docs/database in GitHub.

How you will be graded (100 points = 5 marks)

Criterion	Section	Pts
ER diagram: all entities, keys and relationships with cardinality	Section 2	**30**
Relational schema: fields, types, keys and constraints	Section 3, 4	**30**
Screen-to-data map: every screen has its data	Section 5	**20**
Room plan: entities, DAOs and key queries	Section 6	**10**
GitHub evidence and AI Use Log	Section 7, 8	**10**

Cover Sheet

Team name / number	1	App name	Jeel
GitHub repository link	https://github.com/72ws4c8ngk-beep/Jeel-Mobile-App	Database type	Room (SQLite), schema version 7

#	Student name	Student ID	Role in this milestone
1	Asayel Mohammed Albuainain	2240007410	Data Foundation
2	Ghuyood Abdullah Aldrgam	2240000657	Authentication & Profile
3	Batool Ahmed Alabdullah	2240003056	Mentor Discovery
4	Ragad Saad Al Rowaili	2240004550	Mentor Availability
5	Laila Ali Albadri	2240007284	Booking & My Sessions
6	Aryam Fahad Almarri	2240001503	Mentor Requests & Sessions
7	Luluwah Abdullah Alyousif	2240007777	Session Completion & Feedback

1. Entities at a Glance

Entity (table name)	What it stores	Related to
users	User accounts, login credentials, roles, and profile information	mentors, sessions, notification_reads
mentors	Mentor profiles, specializations, years of experience, and ratings.	users, sessions, availability
sessions	Mentoring session bookings, participants, dates, times, statuses, notes, ratings, and feedback.	users, mentors
availability	Mentor availability slots, including days of the week, start times, and end times.	mentors
notification_reads	Records of notification events marked as read by users.	users
goals	Learning goals created by mentees, including titles, descriptions, and creation timestamps.	users, action_items
action_items	Tasks linked to learning goals, including task titles and completion status.	goals

2. ER Diagram

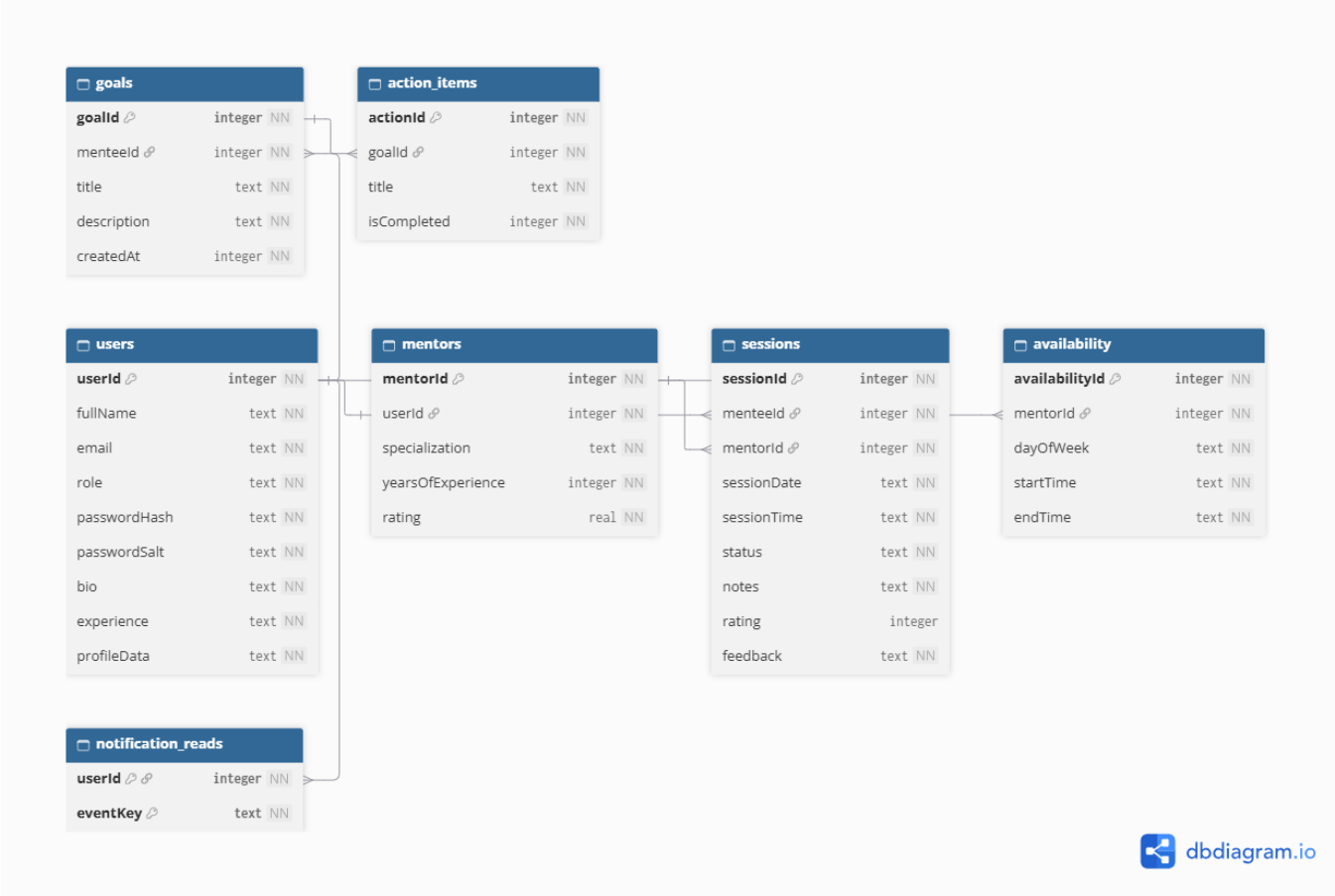


Figure 1. Entity–Relationship Diagram of the Jeel Room Database

3. Relational Schema & Data Dictionary

Table: users

Field	Type	Key	Constraints	Description
userId	INTEGER	PK	NOT NULL; auto-generated	Unique account identifier.
fullName	TEXT		NOT NULL	Name displayed on the profile and in booking details.
email	TEXT		NOT NULL; UNIQUE	Email used to identify the account at sign-in.
role	TEXT		NOT NULL	Account role, Mentor or Mentee.
passwordHash	TEXT		NOT NULL	Hashed password used to verify sign-in credentials.
passwordSalt	TEXT		NOT NULL	Salt used when hashing the account password.
bio	TEXT		NOT NULL	Short biography shown on the profile.
experience	TEXT		NOT NULL	Experience information supplied by the user.
profileData	TEXT		NOT NULL	Additional profile fields stored as JSON text.

Table: mentors

Field	Type	Key	Constraints	Description
mentorId	INTEGER	PK	NOT NULL; auto-generated	Unique mentor profile identifier.
userId	INTEGER	FK	NOT NULL; UNIQUE	References users.userId.
specialization	TEXT		NOT NULL	Mentor’s area of expertise.
yearsOfExperience	INTEGER		NOT NULL	Number of years of professional experience.
rating	REAL		NOT NULL	Average rating from submitted session ratings.

Table: availability

Field	Type	Key	Constraints	Description
availabilityId	INTEGER	PK	NOT NULL; auto-generated	Unique availability slot identifier.
mentorId	INTEGER	FK	NOT NULL	References mentors.mentorId.
dayOfWeek	TEXT		NOT NULL	Weekday on which the mentor is available.

Field	Type	Key	Constraints	Description
startTime	TEXT		NOT NULL	Availability start time stored as validated text, e.g., 3:00 PM.
endTime	TEXT		NOT NULL	Availability end time stored as validated text, e.g., 3:45 PM.

Table: sessions

Field	Type	Key	Constraints	Description
sessionId	INTEGER	PK	NOT NULL; auto-generated	Unique mentorship booking identifier.
menteeId	INTEGER	FK	NOT NULL	References users.userId.
mentorId	INTEGER	FK	NOT NULL	References mentors.mentorId.
sessionDate	TEXT		NOT NULL	Scheduled session date in yyyy-MM-dd format.
sessionTime	TEXT		NOT NULL	Start time copied from the selected availability slot, e.g., 3:00 PM.
status	TEXT		NOT NULL	Current booking status, such as Pending, Accepted, Rejected, or Completed.
notes	TEXT		NOT NULL	Booking details, including the selected topic and optional note.
rating	INTEGER		Nullable	Star rating submitted for a completed session.
feedback	TEXT		NOT NULL	Written review feedback. An empty string indicates no feedback.

Table: notification_reads

Field	Type	Key	Constraints	Description
userId	INTEGER	PK, FK	NOT NULL	References users.userId. Identifies the account that marked the event as read.
eventKey	TEXT	PK	NOT NULL	Identifies the event marked as read for that account.

Table: goals

Field	Type	Key	Constraints	Description
goalId	INTEGER	PK	NOT NULL; auto-generated	Unique learning goal identifier.
menteeId	INTEGER	FK	NOT NULL	References users.userId. Identifies the mentee who owns the goal.
title	TEXT		NOT NULL	Title entered by the mentee.

Field	Type	Key	Constraints	Description
description	TEXT		NOT NULL	Optional goal details. An empty string indicates no description.
createdAt	INTEGER		NOT NULL	Creation time in milliseconds since the Unix epoch.

Composite primary key: (userId, eventKey). Foreign key: ON DELETE CASCADE; ON UPDATE NO ACTION.

Table: action_items

Field	Type	Key	Constraints	Description
actionId	INTEGER	PK	NOT NULL; auto-generated	Unique task identifier.
goalId	INTEGER	FK	NOT NULL	References goals.goalId.
title	TEXT		NOT NULL	Task title entered by the mentee.
isCompleted	INTEGER		NOT NULL	Completion flag stored as 0 for incomplete or 1 for complete.

4. Relationships

Relationship	Type	Implemented by (FK or junction table)	On delete
User account → mentor profile	1 → 0..1	mentors.userId → users.userId (UNIQUE)	CASCADE
Mentee account → sessions	1:N	sessions.menteeId → users.userId	CASCADE
Mentor → sessions	1:N	sessions.mentorId → mentors.mentorId	CASCADE
Mentor → availability slots	1:N	availability.mentorId → mentors.mentorId	CASCADE
User account → read notification events	1:N	notification_reads.userId → users.userId; composite PK: (userId, eventKey)	CASCADE
Mentee account → learning goals	1:N	goals.menteeId → users.userId	CASCADE
Learning goal → action items	1:N	action_items.goalId → goals.goalId	CASCADE

5. Screen-to-Data Map

Screen	Reads (tables / fields)	Writes (insert / update / delete)
Splash / Welcome	—	—
Sign Up / Role Selection	users.email (duplicate check) check)	Insert into users (fullName, email, role, passwordHash, passwordSalt).

Screen	Reads (tables / fields)	Writes (insert / update / delete)
Login	users by email (credentials and role)	—
Forgot Password / Check Email	(UI flow)	—
Profile Onboarding (both roles)	users by email; mentors by userId	Update users; insert/update mentors for mentor onboarding
Mentee Home	Displays mentor profiles from app state and sample content from DemoData.kt.	—
Explore Mentors	mentors JOIN users; name and specialization, rating, profile data); availability	—
Explore Filters	Loaded mentor profiles; filter criteria in UI state	—
Mentor Details	Selected mentor profile; availability by mentorId	—
Book / Reschedule Session	availability by mentorId; sessions (existing booking check)	Inserts a new row into sessions when a booking is confirmed. Rescheduling is not persisted in Room.
Booking Confirmation / Booking Error	Selected booking and UI state	—
Mentee My Sessions	Sessions by menteeId	—
Session Details / Action Plan	Selected sessions record; session state	Update sessions. status for cancellation or completion through applicable actions
Session Review	Sessions (completed session, review state)	Update sessions. rating and sessions. feedback; recalculate mentors. rating
Mentee Progress	Reads goals by menteeId, action_items by goalId, and sessions by menteeId for Your Journey and the completed-session count.	Inserts new records into goals and action_items. Updates action_items.isCompleted.
Notifications (both roles)	Sessions (status and rating); notification_reads by userId	Insert notification_reads for read notifications
My Profile (both roles)	users (profile data); mentors for mentor profiles	—
Edit Mentor / Mentee Profile	users and mentors (current profile)	update users; update mentors for mentor profile
Mentor Home	Displays mentor profile and session information from app state, plus sample content from DemoData.kt.	—

Screen	Reads (tables / fields)	Writes (insert / update / delete)
Mentor Requests / Request Details	Reads sessions by mentorId for pending requests and users.fullName for mentee names. Reads the booking topic and optional note from sessions.notes.	Updates sessions.status to Accepted or Rejected.
Mentor My Sessions	sessions by mentorId; users (mentee names)	Update sessions. status when completing a session
Manage Availability	users by email; mentors by userId; availability by mentorId	Insert, update and delete availability
Settings / Network Error	UI settings or error state	—

6. Room Plan

DAO	Key queries (name + purpose)	Used by (ViewModel / use case)
JeelDao · Users	`getUserByEmail` / `getUserById`: retrieve user accounts; `observeUser`: observe user changes; `insertUser` / `updateUser`: save accounts; `saveProfile`: update user and mentor profiles.	Login, Sign Up, Profile and Edit Profile via `JeelRepository`.
JeelDao · Mentors	`getMentorProfiles`: join users and mentors; `getAllMentors`: observe mentors; `getMentorById` / `getMentorByUserId`: retrieve mentor details; `insertMentor` / `updateMentor`: manage mentor profiles; `refreshMentorRating`: update average rating.	Mentor Discovery, Mentor Details and Profile Management via `JeelRepository`.
JeelDao · Sessions	`insertSession` / `updateSession`: save bookings; `getSessionById`: retrieve session; `getSessionsByMentee` / `getSessionsByMentor`: observe sessions; `countExistingBookings`: prevent booking conflicts; `completeSession` / `cancelSession`: update session status; `submitReview` / `saveReview`: save feedback.	Booking, My Sessions, Mentor Requests, Session Completion and Review via `JeelRepository`.

DAO	Key queries (name + purpose)	Used by (ViewModel / use case)
JeelDao · Availability	`getAvailabilityByMentor`: observe available slots; `insertAvailability` / `updateAvailability` / `deleteAvailability`: manage mentor availability.	Manage Availability, Mentor Details and Booking via `JeelRepository`.
JeelDao · Notifications	`observeReadNotifications: observe read notification keys;` `markNotificationsRead: save notification read status.`	Notifications for both roles via `JeelRepository`.
JeelDao – Goals and Action Items	observeGoals loads the mentee's goals and tasks. insertGoal saves a new goal. addActionForMentee adds a task to a goal owned by the mentee. setActionCompleted updates task completion.	ProgressScreen via JeelRepository.

Entity classes (Kotlin names)	UserEntity, MentorEntity, SessionEntity, AvailabilityEntity, NotificationReadEntity, GoalEntity, and ActionItemEntity. JeelDatabase registers these seven entities using Room (SQLite), database version 7. It includes migrations 1→2, 2→3, 3→4, 4→5, 5→6, and 6→7. The 6→7 migration creates the goals and action_items tables while preserving existing records.
Type converters / enums needed	No custom Room type converters are required. Roles, session statuses, dates, times, and weekdays are stored as String values. Primary and foreign keys use Int. Session ratings use nullable Int values. Goal creation timestamps use Long values, and task completion flags use Boolean values. Room stores both as SQLite INTEGER values.
Sample (seed) data: what and where from	DemoData.kt provides predefined mentor profiles and sample UI content. JeelDatabase does not use an automatic Room seed callback.
Remote source / mock API (if any)	The application uses a local Room (SQLite) database named jeel_database. No remote database or mock API is configured for the Room database.

7. GitHub Evidence

- ☐ Files committed to /docs/database: ER diagram image, schema (this PDF or a .md file)
- ☐ Commit tagged milestone-3-stage-1
- ☐ All Milestone 3-A issues closed; each member has at least one contribution
- ☐ Repository is accessible to the instructor

8. AI Use Log

Tool	Where used	What it was used for	What we changed or checked ourselves
ChatGPT	Android Studio debugging	Helped troubleshoot the Mentee Progress screen and fix how the booking topic and optional notes appear in Request Details.	We applied the changes on our feature branch and tested goal creation, task creation, and task completion in Android Studio.
ChatGPT	Database design and ERD	Helped identify the foreign key relationships linking `goals.menteeId` to `users.userId` and `action_items.goalId` to `goals.goalId`.	We updated the Data Dictionary in Word to document the keys and relationships.

☐ We can explain every table, key and relationship in this design without help.

☐ The design matches our Milestone 2 architecture and navigation.

☐ Table and field names are the same in the diagram, schema and Room plan.